

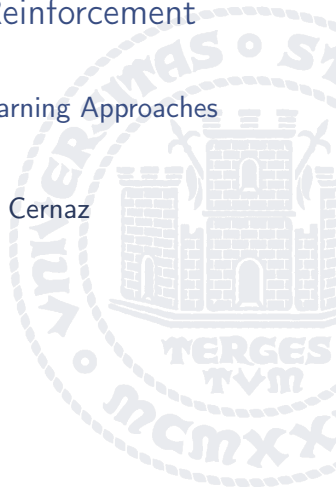
Learning to Play Briscola with Reinforcement Learning

A Comparative Study of Reinforcement Learning Approaches

Alessandro Longato Federico Cernaz

University of Trieste
Reinforcement Learning Course

22 giugno 2026



Project Motivation

Briscola is a popular Italian card game that presents several challenges for Reinforcement Learning:

- ▶ Imperfect information.
- ▶ Sequential decision making.
- ▶ Delayed rewards.
- ▶ Long-term strategic planning.

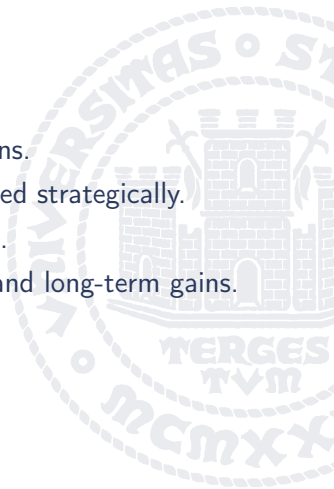
Objective:

Design and train Reinforcement Learning agents capable of learning effective Briscola strategies through interaction and self-play.



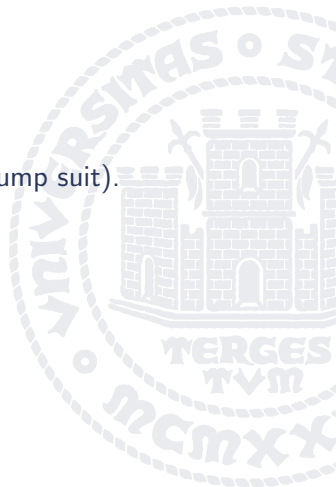
Why is Briscola Challenging?

- ▶ Opponent cards are hidden.
- ▶ Future rewards depend on current decisions.
- ▶ Strong cards may be conserved or sacrificed strategically.
- ▶ Large number of possible game situations.
- ▶ Non-trivial trade-off between short-term and long-term gains.



Briscola Rules

- ▶ Two-player version of Briscola.
- ▶ 40-card Italian deck.
- ▶ One suit is designated as the Briscola (trump suit).
- ▶ Players collect points through tricks.
- ▶ Total points available: 120.
- ▶ Winner: player with more than 60 points.



Custom OpenSpiel Environment

The game was implemented as a custom OpenSpiel environment.

Main components:

- ▶ `BriscolaGame`
- ▶ `BriscolaState`
- ▶ `BriscolaObserver`

Environment properties:

- ▶ Sequential game.
- ▶ Stochastic card dealing.
- ▶ Imperfect information.
- ▶ Zero-sum reward structure.



State Representation (DQN)

The Deep Q-Learning agent receives a fixed-size information-state vector containing only information observable to the acting player.

Feature	Size
Player hand (multi-hot)	40
Briscola suit (one-hot)	4
Current table card (one-hot)	41
Played cards (multi-hot)	40
Own score (normalized)	1
Opponent score (normalized)	1
Remaining deck size	1
Leading player flag	1
Total	129

State Representation (Tabular Q-Learning)

The tabular agent uses a symbolic information-state string that uniquely identifies the observable configuration.

Component	Description
Player ID	Acting player (0 or 1)
Hand	Sorted list of visible cards
Briscola suit	Trump suit index (0–3)
Table card	Card currently on the table

examples

Tabular Q-Learning State Example

P0 | Hand:7,14,33 | Briscola:1 | Table:22

DQN State Example (partial vector)

[0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, ..., 0, 1, 0,
0, 1, 0, ..., 0, 0.25, 0.10, 0.50, 1]

(Full vector has 129 components; only a subset is shown.)

Action Space

Each action corresponds to playing a card from the current hand.

Example:

- ▶ Hand = Ace, Three, King
- ▶ Legal actions = Play Ace, Play Three, Play King

The environment automatically restricts actions to valid cards (which are defined as the cards in the player's hand)

Independent Deep Q-Learning

Two independent DQN agents are trained simultaneously.

Agent 0 (DQN)

vs

Agent 1 (DQN)

Each agent learns its own Q-function while interacting with the other agent.

Deep Q-Network Architecture

Neural network configuration:

- ▶ Input size: 129
- ▶ Hidden Layer 1: 256 neurons
- ▶ Hidden Layer 2: 256 neurons
- ▶ Output size: 40 Q-values

Each output corresponds to the estimated value of playing a specific card.



Training Hyperparameters

- ▶ Replay Buffer: 100,000 transitions
- ▶ Batch Size: 128
- ▶ Learning Rate: 0.001
- ▶ Discount Factor: 1.0
- ▶ Target Network Update: every 500 steps
- ▶ Learning Step: every 64 interactions



Exploration Strategy

An ϵ -greedy policy was used.

- ▶ Initial $\epsilon = 1.0$
- ▶ Final $\epsilon = 0.05$
- ▶ Linear decay over 40

This encourages:

- ▶ Exploration during early training.
- ▶ Exploitation of learned strategies later.



Reward Function

Reward shaping was introduced to provide denser training feedback.

Immediate reward:

$$R_t = 0.2 \times \text{Points obtained from the trick}$$

Terminal reward:

+50 for winning the game

−50 for losing the game

This combines local feedback with the final game outcome.

Why Reward Shaping?

Using only terminal rewards would provide sparse feedback.

Advantages of reward shaping:

- ▶ Faster learning.
- ▶ More informative training signals.
- ▶ Easier credit assignment.
- ▶ Improved convergence stability.



Training Setup

- ▶ Total training episodes: 250,000.
- ▶ Two independent DQN agents.
- ▶ Evaluation every 10,000 episodes.
- ▶ Checkpoint saved every 10,000 episodes.
- ▶ Training can be resumed from saved neural network parameters.



Evaluation Protocol

The trained agent is evaluated against a random opponent.

Evaluation procedure:

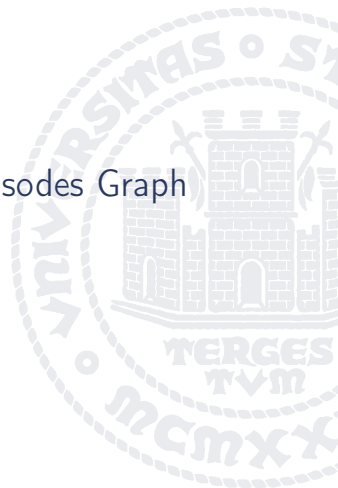
- ▶ 1000 independent games.
- ▶ Exploration disabled.
- ▶ Performance measured using win rate.

$$\text{Win Rate} = \frac{\text{Number of wins}}{\text{Total games}} \times 100$$



Learning Curve

Insert Win Rate vs Training Episodes Graph



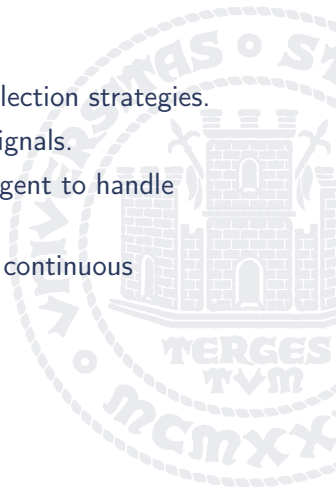
Final Performance

Agent	Win Rate	heightRandom Agent
50IDQN Agent	XX.X	height



Discussion

- ▶ The DQN agents learn non-trivial card selection strategies.
- ▶ Reward shaping provides faster learning signals.
- ▶ The information-state tensor allows the agent to handle imperfect information.
- ▶ Independent agents can improve through continuous interaction.



Conclusions

- ▶ Successfully implemented Briscola in OpenSpiel.
- ▶ Designed a custom information-state representation.
- ▶ Trained agents using Independent Deep Q-Learning.
- ▶ Achieved performance above a random baseline.



Thank You

